Oops & Database Connectivity

🎯 Session Objectives:

- ✅ Understand why database connectivity is important in applications.
- ✅ Explore Python libraries (psycopg2, SQLAlchemy) for connecting with databases.
- ✅ Learn about the connection parameters required to establish a connection.
- ✅ Execute SQL queries (CREATE, SELECT, UPDATE, DELETE, etc.) via Python.
- ✅ Implement real-time examples of database connectivity.

```
# PolyMorphism means -> function with same name but different behaviour.
def sum(val1,val2):
    pass

def sum(val1,val2,val3):
    pass

def sum(*val): # Arbitary Arguments
    return sum(val)
```

```
# Polymorphism 🎭
class SavingsAccount(Account):
    def withdraw(self ,amount , pin):
        if amount > 5000:
            print(f"Savings: Cannot Withdraw more than 5000 at once.")
        else:
            print("Withdraw Successfull ✅")
            super().withdraw(amount , pin) # Use Parent Logic

class CurrentAccount(Account):
    def withdraw(self ,amount , pin):
        fees = 50
        total = fees + amount
        if total > self._balance:
            print(f"Insufficient Funds, Need {total}, but the balance is {self._balance}")
        else:
            print(f"Current: A fee of {fees} will be charged.")
            print("Withdraw Successfull ✅")
            super().withdraw(total, pin) # Parent Logic [Handle]
```

```python
acc_s2 = SavingsAccount('S1001',1111,10000)
acc_s2.withdraw(5000,1111)
acc_s2.check_balance(1111)
```

```
Withdraw Successfull ✅
'Account S1001 having a balance : 5000'
```

```python
acc_c2 = CurrentAccount('C1001',2222,3000)
acc_c2.deposit(2000)
acc_c2.check_balance(2222)
```

```
'Account C1001 having a balance : 5000'
```

```python
acc_c2.withdraw(7000,2222)
acc_c2.check_balance(2222)
```

```
Insufficient Funds, Need 7050, but the balance is 5000
'Account C1001 having a balance : 5000'
```

```python
acc_c2.withdraw(2000,2222)
acc_c2.check_balance(2222)
```

```
Current: A fee of 50 will be charged.
Withdraw Successfull ✅
'Account C1001 having a balance : 2950'
```

```python
# Abstraction -> Defines a Contract & Hide Implementation Details
import abc
dir(abc)
```

```
['ABC',
 'ABCMeta',
 '__builtins__',
 '__doc__',
 '__file__',
 '__loader__',
 '__name__',
 '__package__',
 '__spec__',
 '_abc_init',
 '_abc_instancecheck',
 '_abc_register',
 '_abc_subclasscheck',
 '_get_dump',
 '_reset_caches',
 '_reset_registry',
 'abstractclassmethod',
 'abstractmethod',
 'abstractproperty',
 'abstractstaticmethod',
 'get_cache_token',
 'update_abstractmethods']
```

```python
from abc import ABC , abstractmethod

class AbstractAccount(ABC):

    @abstractmethod
    def deposit(self, amount):
        pass

    @abstractmethod
    def withdraw(self, amount , pin):
        pass

    @abstractmethod
    def check_balance(self, pin):
        pass
```

```python
# Encapsulation [Hide the Sensitive Data] -> Control Access
class Account(AbstractAccount):
    def __init__(self, account_number, pin, balance = 0):
        self.account_number = account_number # Public
        self.__pin = pin # Private
        self._balance = balance # Protected

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
        else:
            print("Deposit Must be Positive")

    def withdraw(self ,amount , pin):
        if pin == self.__pin:
            if amount <= self._balance:
                self._balance -= amount
            else:
                print("Insufficients Funds")
        else:
            print("Invalid PIN")

    def check_balance(self,pin):
        if pin == self.__pin:
            return f"Account {self.account_number} having a balance : {self._balance}"
        else:
            print("Invalid PIN")
```

```python
# Inheritance + Polymorphism 🎉
class SavingsAccount(Account):
    def withdraw(self ,amount , pin):
        if amount > 5000:
            print(f"Savings: Cannot Withdraw more than 5000 at once.")
        else:
            print("Withdraw Successfull ✅")
            super().withdraw(amount , pin) # Use Parent Logic

class CurrentAccount(Account):
    def withdraw(self ,amount , pin):
        fees = 50
        total = fees + amount
        if total > self._balance:
            print(f"Insufficient Funds, Need {total}, but the balance is {self._balance}")
        else:
            print(f"Current: A fee of {fees} will be charged.")
            print("Withdraw Successfull ✅")
            super().withdraw(total, pin) # Parent Logic [Handle]
```

```python
acc_s3 = SavingsAccount('S0003', 1234, 5000)
print(acc_s3.check_balance(1234))
```

```
Account S0003 having a balance : 5000
```

```python
acc_s3.deposit(5000)
print(acc_s3.check_balance(1234))
```

```
Account S0003 having a balance : 10000
```

```python
acc_s3.withdraw(3000,1234)
print(acc_s3.check_balance(1234))
```

```
Withdraw Successfull ✅
Account S0003 having a balance : 7000
```

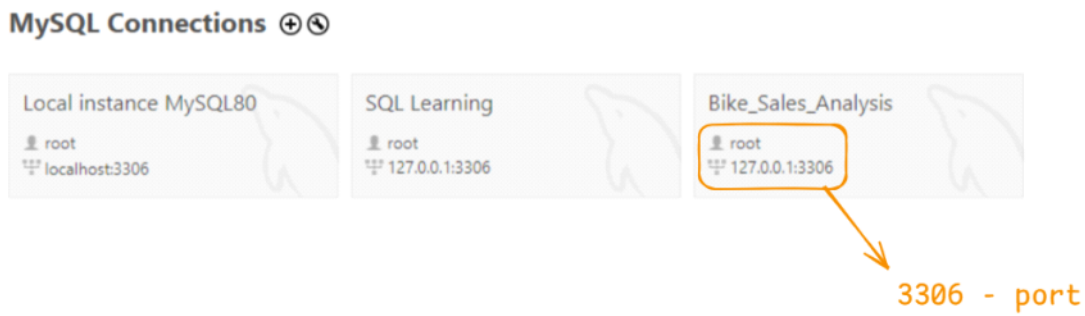
```
# ATM Simulation:
def atm():
    acc1 = SavingsAccount('S101',1111,10000)
    acc2 = SavingsAccount('S102',1234,5000)
    acc3 = SavingsAccount('S103',2222,7000)
    acc4 = SavingsAccount('S104',5678,9000)
    acc5 = SavingsAccount('S105',0000,11000)

    accounts = {
        'S101' : acc1,
        'S102' : acc2,
        'S103' : acc3,
        'S104' : acc4,
        'S105' : acc5,
    }

    while True:
        print("\n 1. Deposit \n 2. Withdraw \n 3. Check Balance \n 4. Exit")
        choice = int(input("Enter the Choice: "))
        if choice == 4:
            return
        acc_num = input("Enter your Account Number: ")
        pin = int(input("Enter your ATM PIN: "))

        if acc_num in accounts:
            acc = accounts[acc_num] # Fetching the Object

            if choice == 1: # Deposit
                amt = int(input("Enter the amount to Deposit:"))
                acc.deposit(amt)
            elif choice == 2: # Withdraw
                amt = int(input("Enter the amount to Withdraw:"))
                acc.withdraw(amt,pin)
            elif choice == 3: # Check Balance
                print(acc.check_balance(pin))
            else:
                print("Invalid Choice")
        else:
            print("Account Not Found!")

atm()
```

```
 1. Deposit
 2. Withdraw
 3. Check Balance
 4. Exit
Enter the Choice:  1
Enter your Account Number:  S104
Enter your ATM PIN:  5678
Enter the amount to Deposit: 2000

 1. Deposit
 2. Withdraw
 3. Check Balance
 4. Exit
Enter the Choice:  3
Enter your Account Number:  S104
Enter your ATM PIN:  5678
Account S104 having a balance : 11000

 1. Deposit
 2. Withdraw
 3. Check Balance
 4. Exit
Enter the Choice:  2
Enter your Account Number:  S104
Enter your ATM PIN:  5678
Enter the amount to Withdraw: 4000
Withdraw Successfull ✅
```

```
 1. Deposit
 2. Withdraw
 3. Check Balance
 4. Exit
Enter the Choice:  3
Enter your Account Number:  S104
Enter your ATM PIN:  5678
Account S104 having a balance : 7000

 1. Deposit
 2. Withdraw
 3. Check Balance
 4. Exit
Enter the Choice:  1
Enter your Account Number:  S104
Enter your ATM PIN:  5678
Enter the amount to Deposit: -1000
Deposit Must be Positive

 1. Deposit
 2. Withdraw
 3. Check Balance
 4. Exit
Enter the Choice:  4
```

Database Connectivity

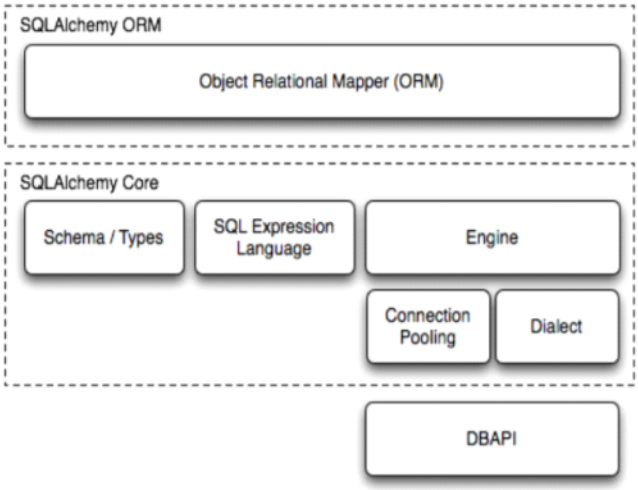
```python
import mysql.connector
# Example for MySQL
connection = mysql.connector.connect(
    host="localhost",
    user="your_user",
    password="your_password",
    port = 'your_port',
    database="your_database"
)
```



What is SQLAlchemy?

SQLAlchemy is a Python toolkit and Object-Relational Mapper (ORM) for working with relational databases like MySQL, PostgreSQL, SQLite, and more.

```
# Format: dialect+driver://username:password@host:port/database
```



```
pip install mysql-connector-python
```

```
Requirement already satisfied: mysql-connector-python in c:\users\krish\.anaconda\new.anaconda\lib\site-packag
es (9.4.0)Note: you may need to restart the kernel to use updated packages.
```

```
import mysql.connector
# Example for MySQL
cnx = mysql.connector.connect(
    host="localhost",
    user="root",
    password="1234567890",
    port = '3306',
    database="weekend_analysis"
)
if cnx.is_connected():
    print("Connected to the weekend_analysis database ✅")
```

```
Connected to the weekend_analysis database ✅
```

```
cur = cnx.cursor(buffered = True)
cur.execute("USE Weekend_Analysis;")
cur.execute("DROP TABLE IF EXISTS Students;")
```

```python
# Lets Create a Student Table From Fresh
cur.execute(
    """
       CREATE TABLE Students(
            student_id INT AUTO_INCREMENT,
            name VARCHAR(50),
            age INT,
            grade VARCHAR(5),
            PRIMARY KEY(student_id)
       );
    """
)
```

```python
# Describe the Students Table
cur.execute("DESC Students;")
print(cur.fetchall())
```

```
[('student_id', 'int', 'NO', 'PRI', None, 'auto_increment'), ('name', 'varchar(50)', 'YES', '', None, ''), ('a
ge', 'int', 'YES', '', None, ''), ('grade', 'varchar(5)', 'YES', '', None, '')]
```

```python
cur.execute("DESC Students;")
rows = cur.fetchall()
for row in rows:
    print(row)
```

```
('student_id', 'int', 'NO', 'PRI', None, 'auto_increment')
('name', 'varchar(50)', 'YES', '', None, '')
('age', 'int', 'YES', '', None, '')
('grade', 'varchar(5)', 'YES', '', None, '')
```

```python
cur.description
```

```
[('Field', 253, None, None, None, None, 1, 0, 255),
 ('Type', 252, None, None, None, None, 0, 4241, 255),
 ('Null', 253, None, None, None, None, 0, 1, 255),
 ('Key', 254, None, None, None, None, 0, 4481, 255),
 ('Default', 252, None, None, None, None, 1, 144, 255),
 ('Extra', 253, None, None, None, None, 1, 0, 255)]
```

```python
for col in cur.description:
    print(col[0])
```

```
Field
Type
Null
Key
Default
Extra
```

```python
headers = [col[0] for col in cur.description] # List Comprehension
headers
```

```
['Field', 'Type', 'Null', 'Key', 'Default', 'Extra']
```

```python
headers = []
for col in cur.description:
    headers.append(col[0])
headers
```

```
['Field', 'Type', 'Null', 'Key', 'Default', 'Extra']
```

```python
headers = []
for col in cur.description:
    headers.append(col[0])
headers
```

```
['Field', 'Type', 'Null', 'Key', 'Default', 'Extra']
```

```python
cur.execute("DESC Students;")
rows = cur.fetchall()

# Converting the raw info into DataFrame
import pandas as pd
df = pd.DataFrame(rows, columns = headers)
df
```

	Field	Type	Null	Key	Default	Extra
0	student_id	int	NO	PRI	None	auto_increment
1	name	varchar(50)	YES		None	
2	age	int	YES		None	
3	grade	varchar(5)	YES		None	

```python
# Insert Multiple Records on Students Table
cur.execute(
    """
     INSERT INTO Students(name,age,grade)
     VALUES('Deepak',28,'A+'),
     ('Shyam',26,'A'),
     ('Ajay',30,'B'),
     ('Rohan',29,'C'),
     ('Shahzain',27,'A'),
     ('Rahul', 30 ,'B'),
     ('Shruti',28,'B+');
    """
)
```

```python
# Reading out the Students Table
cur.execute("SELECT * FROM Students;")
for rows in cur.fetchall():
    print(rows)
```

```
(1, 'Deepak', 28, 'A+')
(2, 'Shyam', 26, 'A')
(3, 'Ajay', 30, 'B')
(4, 'Rohan', 29, 'C')
(5, 'Shahzain', 27, 'A')
(6, 'Rahul', 30, 'B')
(7, 'Shruti', 28, 'B+')
```

```python
# cnx.commit() -> will commit the records.
# System Crash
cur.close()
cnx.close()
```

```python
import mysql.connector
# Example for MySQL
cnx = mysql.connector.connect(
    host="localhost",
    user="root",
    password="1234567890",
    port = '3306',
    database="weekend_analysis"
)
if cnx.is_connected():
    print("Connected to the weekend_analysis database ✅")
```

```
Connected to the weekend_analysis database ✅
```

```python
cur = cnx.cursor(buffered = True)
cur.execute("SELECT * FROM Students;")
print(cur.fetchall())
```

```
[]
```

```python
# Insert Multiple Records on Students Table
cur.execute(
    """
     INSERT INTO Students(name,age,grade)
     VALUES('Deepak',28,'A+'),
     ('Shyam',26,'A'),
     ('Ajay',30,'B'),
     ('Rohan',29,'C'),
     ('Shahzain',27,'A'),
     ('Rahul', 30 ,'B'),
     ('Shruti',28,'B+');
    """
)
```

```python
cnx.commit()
```

```python
# Reading out the Students Table
cur.execute("SELECT * FROM Students;")
for rows in cur.fetchall():
    print(rows)
```

```
(8, 'Deepak', 28, 'A+')
(9, 'Shyam', 26, 'A')
(10, 'Ajay', 30, 'B')
(11, 'Rohan', 29, 'C')
(12, 'Shahzain', 27, 'A')
(13, 'Rahul', 30, 'B')
(14, 'Shruti', 28, 'B+')
```

```python
# Closing the Cursor and Connection after commiting the insert data [Saved]
cur.close()
cnx.close()
```

```python
import mysql.connector
# Example for MySQL
cnx = mysql.connector.connect(
    host="localhost",
    user="root",
    password="1234567890",
    port = '3306',
    database="weekend_analysis"
)
if cnx.is_connected():
    print("Connected to the weekend_analysis database ✅")
```

```
Connected to the weekend_analysis database ✅
```

```python
cur = cnx.cursor(buffered = True)
cur.execute("SELECT * FROM Students;")
for rows in cur.fetchall():
    print(rows)
```

```
(8, 'Deepak', 28, 'A+')
(9, 'Shyam', 26, 'A')
(10, 'Ajay', 30, 'B')
(11, 'Rohan', 29, 'C')
(12, 'Shahzain', 27, 'A')
(13, 'Rahul', 30, 'B')
(14, 'Shruti', 28, 'B+')
```

```python
stud_headers = df['Field'].values
stud_headers
```

```
array(['student_id', 'name', 'age', 'grade'], dtype=object)
```

```python
# Reading the above data via DataFrame
cur.execute("SELECT * FROM Students;")
rows = cur.fetchall()

# DataFrame
import pandas as pd
students_df = pd.DataFrame(rows , columns = stud_headers)
students_df
```

	student_id	name	age	grade
0	8	Deepak	28	A+
1	9	Shyam	26	A
2	10	Ajay	30	B
3	11	Rohan	29	C
4	12	Shahzain	27	A
5	13	Rahul	30	B
6	14	Shruti	28	B+

```python
cur.execute("SELECT * FROM Customers;")
for rows in cur.fetchall():
    print(rows)
```

```
(11000, 'MR.', 'JON', 'YANG', 'JON YANG', datetime.date(1966, 8, 4), 'M', 'jon24@learnsector.com', 'M', Non
e, 90000, 'Moderate Income', 2, 'Bachelors', 'Professional', 'Y', 6697921661, 'China')
(11001, 'MR.', 'EUGENE', 'HUANG', 'EUGENE HUANG', datetime.date(1965, 5, 14), 'S', 'eugene10@learnsector.co
m', 'M', None, 60000, 'Moderate Income', 3, 'Bachelors', 'Professional', 'N', 8586376955, 'Japan')
(11002, 'MR.', 'RUBEN', 'TORRES', 'RUBEN TORRES', datetime.date(1965, 12, 8), 'M', 'ruben35@learnsector.co
m', 'M', None, 60000, 'Moderate Income', 3, 'Bachelors', 'Professional', 'Y', 6528852745, 'Japan')
(11003, 'MS.', 'CHRISTY', 'ZHU', 'CHRISTY ZHU', datetime.date(1968, 2, 15), 'S', 'christy12@learnsector.co
m', 'F', None, None, 'No Income Defined', 0, 'Bachelors', 'Professional', 'N', 3646568427, 'Japan')
(11004, 'MRS.', 'ELIZABETH', 'JOHNSON', 'ELIZABETH JOHNSON', datetime.date(1968, 8, 8), 'S', 'elizabeth5@lea
rnsector.com', 'F', None, 80000, 'Moderate Income', 5, 'Bachelors', 'Professional', 'Y', 7132265883, 'Japa
n')
(11005, 'MR.', 'JULIO', 'RUIZ', 'JULIO RUIZ', datetime.date(1965, 5, 8), 'S', 'julio1@learnsector.com', 'M',
None, 70000, 'Moderate Income', 0, 'Bachelors', 'Professional', 'Y', 1902716743, 'Japan')
(11007, 'MR.', 'MARCO', 'MEHTA', 'MARCO MEHTA', datetime.date(1964, 9, 5), 'M', 'marco14@learnsector.com',
'M', None, 60000, 'Moderate Income', 3, 'Bachelors', 'Professional', 'Y', 4203325209, 'Japan')
(11008, 'MRS.', 'ROBIN', 'VERHOFF', 'ROBIN VERHOFF', datetime.date(1964, 7, 7), 'S', 'rob4@learnsector.com',
'F', None, 60000, 'Moderate Income', 4, 'Bachelors', 'Professional', 'Y', 2866694333, 'Japan')
```

```python
cur.execute("DESC Customers")
customers_headers = []
for rows in cur.fetchall():
    customers_headers.append(rows[0])

print(customers_headers)
```

```
['CustomerKey', 'Prefix', 'FirstName', 'LastName', 'FullName', 'DateOfBirth', 'MaritalStatus', 'EmailAddress',
'Gender', 'Regions', 'AnnualIncome', 'IncomeCategory', 'TotalChildren', 'EducationLevel', 'Occupation', 'HomeO
wner', 'Phone_number', 'Country']
```

```python
# Reading the above data via DataFrame
cur.execute("SELECT * FROM Customers;")
rows = cur.fetchall()

# DataFrame
import pandas as pd
customers = pd.DataFrame(rows , columns = customers_headers)
customers
```

	CustomerKey	Prefix	FirstName	LastName	FullName	DateOfBirth	MaritalStatus	EmailAddress	Gender
0	11000	MR.	JON	YANG	JON YANG	1966-08-04	M	jon24@learnsector.com	M
1	11001	MR.	EUGENE	HUANG	EUGENE HUANG	1965-05-14	S	eugene10@learnsector.com	M
2	11002	MR.	RUBEN	TORRES	RUBEN TORRES	1965-12-08	M	ruben35@learnsector.com	M
3	11003	MS.	CHRISTY	ZHU	CHRISTY ZHU	1968-02-15	S	christy12@learnsector.com	F
4	11004	MRS.	ELIZABETH	JOHNSON	ELIZABETH JOHNSON	1968-08-08	S	elizabeth5@learnsector.com	F
...	...	...	...	...	...	...	...	...	...
2057	13094	MR.	ANTONIO	DIAZ	ANTONIO DIAZ	1957-03-23	M	antonio22@learnsector.com	M
2058	13095	MR.	JORDAN	GRIFFIN	JORDAN GRIFFIN	1957-07-24	S	jordan18@learnsector.com	M
2059	13096	MR.	JONATHAN	CHEN	JONATHAN CHEN	1957-10-20	M	jonathan24@learnsector.com	M
2060	13097	MR.	DALTON	HUGHES	DALTON HUGHES	1957-05-14	M	dalton57@learnsector.com	M
2061	13098	MR.	MIGUEL	RUSSELL	MIGUEL RUSSELL	1957-07-10	M	miguel66@learnsector.com	M

2062 rows × 18 columns

```python
cur.execute("DESC Products")
products_headers = []
for rows in cur.fetchall():
    products_headers.append(rows[0])

print(products_headers)
```

```
['ProductKey', 'ProductSubcategoryKey', 'ProductSKU', 'ProductName', 'ModelName', 'ProductDescription', 'Produ
ctColor', 'ProductSize', 'ProductStyle', 'ProductCost', 'ProductPrice']
```

```python
# Reading the above data via DataFrame
cur.execute("SELECT * FROM Products;")
rows = cur.fetchall()

# DataFrame
import pandas as pd
products = pd.DataFrame(rows , columns = products_headers)
products
```

	ProductKey	ProductSubcategoryKey	ProductSKU	ProductName	ModelName	ProductDescription	ProductColor	ProductS
0	214	31.0	HL-U509-R	Sport-100 Helmet, Red	Sport-100	Universal fit, well-vented, lightweight , snap...	Red	
1	215	31.0	HL-U509	Sport-100 Helmet, Black	Sport-100	Universal fit, well-vented, lightweight , snap...	Black	
2	218	23.0	SO-B909-M	Mountain Bike Socks, M	Mountain Bike Socks	Combination of natural and synthetic fibers st...	White	
3	219	23.0	SO-B909-L	Mountain Bike Socks, L	Mountain Bike Socks	Combination of natural and synthetic fibers st...	White	

```python
# For Every Product, Find out the Total Revenue
cur.execute(
    """
        WITH SummaryTable AS (
            SELECT
                s.ProductKey,
                ROUND(SUM(p.ProductPrice * s.OrderQuantity),0) AS TotalRevenue
            FROM Sales2017 s
            JOIN Products p
            ON p.ProductKey = s.ProductKey
            GROUP BY 1
        )
        SELECT
            p.ProductKey,
            p.ProductName,
            s.TotalRevenue
        FROM Products p
        JOIN SummaryTable s
        ON p.ProductKey = s.ProductKey
        ORDER BY 3 DESC;
    """
)
rows = cur.fetchall()

# DataFrame
headers = ['ProductKey' , 'ProductName' , 'TotalRevenue']
product_wise_revenue = pd.DataFrame(rows , columns = headers)
product_wise_revenue
```

	ProductKey	ProductName	TotalRevenue
0	358	Mountain-200 Black, 38	514324.0
1	352	Mountain-200 Silver, 38	513712.0
2	356	Mountain-200 Silver, 46	505426.0
3	360	Mountain-200 Black, 42	504078.0
4	354	Mountain-200 Silver, 42	466069.0
...	...	...	...
97	473	Classic Vest, L	7302.0
98	472	Classic Vest, M	7048.0
99	471	Classic Vest, S	5778.0
100	482	Racing Socks, L	2643.0
101	481	Racing Socks, M	2571.0

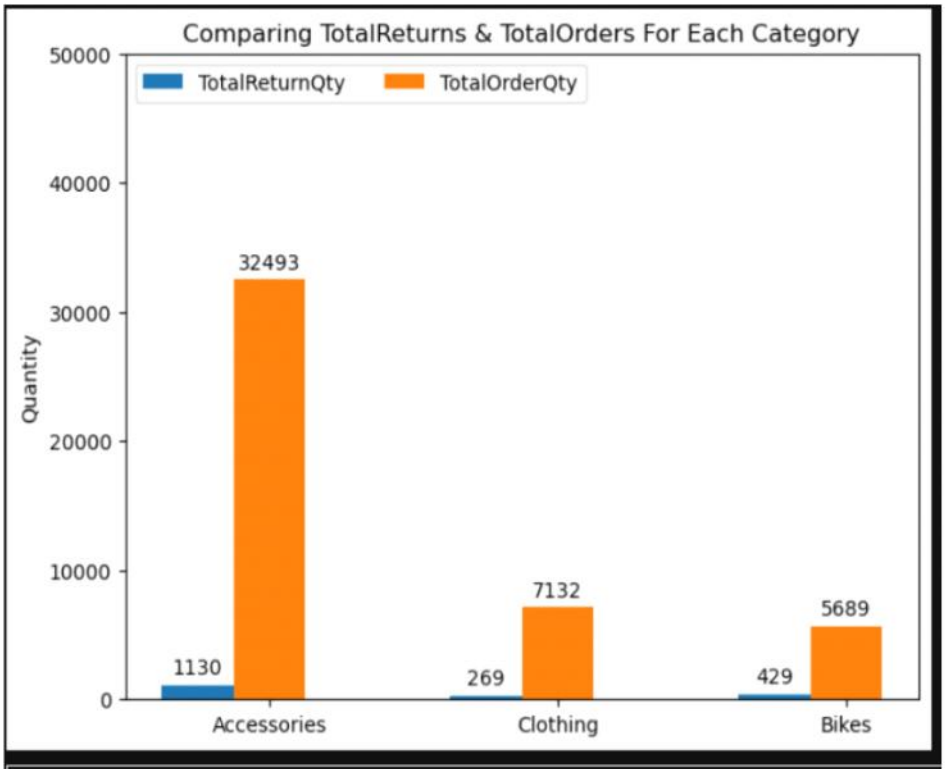
102 rows × 3 columns

```
# For Every Product, Find out the Total Revenue
cur.execute(
    """
        With CategoryReturns AS (
            Select
                pc.CategoryName,
                SUM(r.ReturnQuantity) AS TotalReturnQty
            FROM ProductCategories pc
            JOIN ProductSubcategories ps
            ON pc.ProductCategoryKey = ps.ProductCategoryKey
            JOIN Products p
            ON p.ProductSubcategoryKey = ps.ProductSubcategoryKey
            JOIN Returns r
            ON r.ProductKey = p.ProductKey
            GROUP BY 1
        ),
        CategorySales AS (
            Select
                pc.CategoryName,
                ROUND(SUM(s.OrderQuantity),0) AS TotalOrderQty
            FROM ProductCategories pc
            JOIN ProductSubcategories ps
            ON pc.ProductCategoryKey = ps.ProductCategoryKey
            JOIN Products p
            ON p.ProductSubcategoryKey = ps.ProductSubcategoryKey
            JOIN Sales2017 s
            ON s.ProductKey = p.ProductKey
            GROUP BY 1
        )
        SELECT
            cr.CategoryName,
            cr.TotalReturnQty
```

```
        SELECT
            cr.CategoryName,
            cr.TotalReturnQty,
            cs.TotalOrderQty
        FROM CategoryReturns cr
        JOIN CategorySales cs
        ON cr.CategoryName = cs.CategoryName
        ORDER BY cs.TotalOrderQty DESC;
    """
)
rows = cur.fetchall()

# DataFrame
headers = ['CategoryName' , 'TotalReturnQty' , 'TotalOrderQty']
product_wise_revenue = pd.DataFrame(rows , columns = headers)
product_wise_revenue
```

	CategoryName	TotalReturnQty	TotalOrderQty
0	Accessories	1130	32493
1	Clothing	269	7132
2	Bikes	429	5689

```
import matplotlib.pyplot as plt
import numpy as np

CategoryName = ("Accessories", "Clothing", "Bikes")
data_insights = {
    'TotalReturnQty': (1130, 269, 429),
    'TotalOrderQty': (32493,7132,5689),
}

x = np.arange(len(CategoryName))  # the label locations
width = 0.25  # the width of the bars
multiplier = 0

fig, ax = plt.subplots(layout='constrained')

for attribute, val in data_insights.items():
    offset = width * multiplier
    rects = ax.bar(x + offset, val, width, label=attribute)
    ax.bar_label(rects, padding=3)
    multiplier += 1

# Add some text for labels, title and custom x-axis tick labels, etc.
ax.set_ylabel('Quantity')
ax.set_title('Comparing TotalReturns & TotalOrders For Each Category')
ax.set_xticks(x + width, CategoryName)
ax.legend(loc='upper left', ncols=3)
ax.set_ylim(0, 50000)

plt.show()
```




```
# Closing the cursor and connection final time.
cur.close()
cnx.close()
```

```
# SQLAlchemy [ORM] [Object Relational Mapper]
from sqlalchemy import create_engine
# Format: dialect+driver://username:password@host:port/database
engine = create_engine("mysql+pymysql://root:1234567890@localhost:3306/weekend_analysis")
```

```
# Test Connection
from sqlalchemy import text

with engine.connect() as conn: # Turn off the engine after being done.
    result = conn.execute(
        text(
            "SELECT DATABASE();"
        )
    )
    print(result.fetchall())
```

```
[('weekend_analysis',)]
```

```
category_insights = pd.read_sql(
    text(
        """
            With CategoryReturns AS (
                Select
                    pc.CategoryName,
                    SUM(r.ReturnQuantity) AS TotalReturnQty
                FROM ProductCategories pc
                JOIN ProductSubcategories ps
                ON pc.ProductCategoryKey = ps.ProductCategoryKey
                JOIN Products p
                ON p.ProductSubcategoryKey = ps.ProductSubcategoryKey
                JOIN Returns r
                ON r.ProductKey = p.ProductKey
                GROUP BY 1
            ),
            CategorySales AS (
                Select
                    pc.CategoryName,
                    ROUND(SUM(s.OrderQuantity),0) AS TotalOrderQty
                FROM ProductCategories pc
                JOIN ProductSubcategories ps
                ON pc.ProductCategoryKey = ps.ProductCategoryKey
                JOIN Products p
                ON p.ProductSubcategoryKey = ps.ProductSubcategoryKey
                JOIN Sales2017 s
                ON s.ProductKey = p.ProductKey
                GROUP BY 1
            )
            SELECT
                cr.CategoryName,
                cr.TotalReturnQty,
                cs.TotalOrderQty
            FROM CategoryReturns cr
            JOIN CategorySales cs
            ON cr.CategoryName = cs.CategoryName
            ORDER BY cs.TotalOrderQty DESC;
        """
    ),engine
)
category_insights
```

	CategoryName	TotalReturnQty	TotalOrderQty
0	Accessories	1130.0	32493.0
1	Clothing	269.0	7132.0
2	Bikes	429.0	5689.0

```
avgprice_prod = pd.read_sql(
    text(
        """
            WITH AvgProductSubcategory AS (
                SELECT
                    ProductSubcategoryKey,
                    ROUND(AVG(ProductPrice),0) AS AvgPrice
                FROM Products
                GROUP BY 1
            )
            SELECT
                ProductKey,
                ProductName,
                ProductPrice,
                AvgPrice
            FROM Products p
            JOIN AvgProductSubcategory a
            ON p.ProductSubcategoryKey = a.ProductSubcategoryKey
            WHERE ProductPrice > AvgPrice
            ORDER BY ProductPrice DESC;
        """
    ),engine
)
avgprice_prod
```

	ProductKey	ProductName	ProductPrice	AvgPrice
0	310	Road-150 Red, 62	3578.27	1530.0
1	311	Road-150 Red, 44	3578.27	1530.0
2	312	Road-150 Red, 48	3578.27	1530.0
3	313	Road-150 Red, 52	3578.27	1530.0
4	314	Road-150 Red, 56	3578.27	1530.0
...	...	...	...	...
113	559	Chain	20.24	20.0
114	478	Mountain Bottle Cage	9.99	8.0
115	218	Mountain Bike Socks, M	9.50	9.0
116	219	Mountain Bike Socks, L	9.50	9.0
117	479	Road Bottle Cage	8.99	8.0

118 rows × 4 columns

```
revenue_category = pd.read_sql(
    text(
        """
            SELECT
                pc.CategoryName,
                t.Region,
                ROUND(SUM(p.ProductPrice * s.OrderQuantity),0) AS TotalRevenue,
                CASE
                    WHEN ROUND(SUM(p.ProductPrice * s.OrderQuantity),0) > 200000 THEN 'High Revenue'
                    WHEN ROUND(SUM(p.ProductPrice * s.OrderQuantity),0) > 50000 THEN 'Moderate Revenue'
                    ELSE 'Low Revenue'
                END AS RevenueCategory
            FROM ProductCategories pc
            JOIN ProductSubcategories ps
            ON pc.ProductCategoryKey = ps.ProductCategoryKey
            JOIN Products p
            ON p.ProductSubcategoryKey = ps.ProductSubcategoryKey
            JOIN Sales2017 s
            ON s.ProductKey = p.ProductKey
            JOIN Territories t
            ON t.SalesTerritoryKey = s.TerritoryKey
            GROUP BY 1,2;
        """
    ), engine
)
revenue_category
```

	CategoryName	Region	TotalRevenue	RevenueCategory
0	Accessories	Northwest	76725.0	Moderate Revenue
1	Bikes	Northwest	1096794.0	High Revenue
2	Accessories	Southwest	106170.0	Moderate Revenue
3	Clothing	Southwest	46631.0	Low Revenue
4	Accessories	Canada	70377.0	Moderate Revenue
5	Accessories	United Kingdom	55848.0	Moderate Revenue
6	Clothing	United Kingdom	19433.0	Low Revenue
7	Accessories	Germany	47030.0	Low Revenue
8	Bikes	Germany	953155.0	High Revenue

```python
# Top 10 Products based on Total Revenue

top_10_prod_insight = pd.read_sql(
    text(
        """
            WITH AllSales AS(
                SELECT * FROM Sales2015
                UNION
                SELECT * FROM Sales2016
                UNION
                SELECT * FROM Sales2017
            ) -- 56046 row(s) returned
            SELECT
                p.ProductName,
                ROUND(SUM(s.OrderQuantity * p.ProductPrice),0) AS TotalRevenue,
                ROUND(SUM(s.OrderQuantity * p.ProductCost),0) AS TotalExpenses
            FROM AllSales s
            JOIN Products p
            ON p.ProductKey = s.ProductKey
            GROUP BY 1
            ORDER BY 2 DESC
            LIMIT 10;
        """
    ), engine
)
top_10_prod_insight
```

	ProductName	TotalRevenue	TotalExpenses
0	Mountain-200 Black, 46	1241754.0	670121.0
1	Mountain-200 Black, 42	1233557.0	665698.0
2	Mountain-200 Silver, 38	1213852.0	655064.0
3	Mountain-200 Silver, 46	1182781.0	638296.0
4	Mountain-200 Black, 38	1165937.0	629206.0
5	Mountain-200 Silver, 42	1133067.0	611467.0
6	Road-250 Black, 52	689374.0	417336.0
7	Road-250 Red, 58	661013.0	400167.0
8	Road-250 Black, 48	641379.0	388281.0
9	Road-150 Red, 48	640510.0	388662.0